

1) Diff between List & Tuple

→ List Tuple

- | | |
|--|---|
| - Mutable (editable) | - Immutable |
| - Loops with list are time consuming | - Compared to list loops in tuples are faster |
| - Better for insertion & Deletion operations | - Accessing elements efficiently in tuples is more feasible |
| - More memory | - Less memory |
| - More built-in methods | - Fewer built-in methods |
| - More prone to errors & changes | - Less room for error |

2) Difference between == & is in

→ Equality Operator (==) It checks if 2 objects are of same value irrespective of memory location
is operator It checks for same memory location irrespective of value, values could be different even if the operation being performed

3) Mutable & Immutable in Python

→ Mutable - These can be modified after creation, eg. list, dict, set

Immutable - These can't be modified after creation eg. int, float, bool, str, tuple

4) Purpose of self?

- One of the fundamental concepts of OOP
- It refers to the current object instance
- Can be used to access attribute, method, within class

5) Diff between Instance, class & static

- Static - This method belongs to the class rather than any specific object, allowing it to be called without creating an instance
- Class - Methods are functions that belong to a class.

Instance - It belongs to an object of a class & is used to operate on object specific data

6) *args & **kwargs diff?

- *args (Arbitrary arguments) - Number of arguments can be passed into your function, ~~before the parameter name~~
- **kwargs (Arbitrary keyword Arguments) - Number of keyword arguments can be passed into the function

7) Decorator & why it's used?

- An easier way to modify or extend the behavior of function without ever changing the actual code

eg. def func():

Print("Hello")

def decorator(f):

def wrapper():

Print("Before") f()

return wrapper

11) Lambda?

- - A small anonymous function.
- It could take any number of arguments, but can only have one expression

- eg. `def myfunc(n):`

`return lambda a: a*n`

`mydoubler = myfunc(2)`

`Print (myfunc(2))` → lambda object

`Print (mydoubler(11))` → 22

12) Map(), filter() & reduce()

→ `Map()` - returns a map object of the results after applying the given function to each item of a given iterable

`Reduce()` - To apply particular function passed in its arguments to all of list elements mentioned in the sequence passed along.

`filter()` - Filters the given sequence with help of function that tests each element in the sequence to be true or not

eg. `n = [1, 2, 3, 4, 5]`

`def double(n):`

`return n*2`

`print (map(double, n))`

`[2, 4, 6, 8, 10]`

`import functools`

`print (functools.reduce`

`(lambda x, y: x*y, n))`

`Result: 120`

`def is-even(n):`

`return n%2==0`

`print (list`

`filter(is-even, n))`

`[2, 4]`

13) Closure?

→ These functions allow a function to remember values from the environment in which it was created even if that environment no longer exist

- Also known as **memory-equipped functions**

eg.

```
def outer_fn(x):  
    def inner_fn(y):  
        return x+y  
    return inner_fn
```

closure = outer_fn(10)

print(closure(5)) → 15

print(closure(10)) → 20

14) Functional programming?

→ - It decomposes a problem into set of functions
- Takes input & produce output, doesn't have any internal state that affect the output produced.

15) Inheritance & types?

→ Allows to define class that inherits all methods & properties from another class

Single

Multiple

Multilevel

Hierarchical

A

A

B

A

A

↓

↘ ↙

↓

↙ ↓ ↘

B

C

B

B

C

D

↓
C

16) Polymorphism & type supported by python?

- The ability of variable object or function to take on the multiple forms.
- Runtime overriding when a child class method redefines a method that already exist in parent class

17) Encapsulation?

- Wrapping of data, methods & properties into single unit.
- Violation encapsulation can be done when a class field is accessed without a property.

18) Abstraction?

- Showing only required things & hiding the background details.

eg- from abc import ABC, abstractmethod

```
class Greet(ABC):  
    @abstractmethod  
    def say_hello(self):  
        pass  
class English(Greet):  
    def say_hello(self):  
        return
```

g = English()
print(g.say_hello())
→ Hello!

19) Super()?

- It's a way to access methods from parent class without hard coding parent class name, it follows MRO

20) Dunder methods?

- Special methods with double `--` that enable operator overloading & custom object behavior
 - Built for every object
 - Invoked implicitly
 - Customizable
- } make classes callable for simplicity & lower

21) GIL & Multithreading Effect?

- Problem - Ref count variable needed protection from race condition where 2 threads increase or decrease its value simultaneously; if this happens it can cause memory leak or incorrectly release the memory while ref object still exist

GIL - A single lock on the interpreter itself which adds a rule that execution of any bytecode requires acquiring the interpreter lock, prevents **dead lock**

Multi-threading Effect GIL prevents CPU bound threads from executing in parallel, the I/O bound program won't have much of performance impact, but CPU bound would be forced to use single thread which will increase execution time.

22) Parallelism ?

- - Imp to divide the problem into sub-units that don't depend other sub-units
 - A problem where the sub-units are totally independent of other sub-unit is called **Embarrassingly parallel**.
 - **Python Parallel framework** - this package provides a framework to setup & execute a task on single, multi-core machines & multiple nodes, connected to a network.
 - **Engines** are set of workers **Controller** is the manager

23) Multi-threading, multi-processing, concurrency & parallelism

- **Concurrency** Structuring a program to handle multiple task in overlapping time period
- Parallelism** Executing multiple computational tasks at exact same time

Multi-threading Running multiple thread within one process

Multiprocessing Running multiple independent processes

24) When to use thread, process & async/await

- - Depends on program is waiting on CPU or waiting on I/O & whether external libraries support asynchronous code

Tool	Concurrency type	Best for	Max scale	Shared mem
async/await	Cooperative (single thread)	High volume network/ API tasks	Tens of thousands	Yes (no lock needed)
Threads	Preemptive (OS-level)	legacy I/O, file system scrapping	Hundreds	Yes (requires lock)
Processes	Parallel	CPU-heavy tasks data processing	Number of CPUs	No

25) `--init__()` ?

→ A constructor which is automatically called when a new object or a class is created.

26) key error in python ?

→ It occurs when you use a key on a dictionary & key doesn't exist
- Using try - ~~catch~~^{except} block to handle key error.

27) Memory management in python ?

→ - A process to allocate & deallocate memory to program while it runs.
- Python handles this automatically with reference counting & garbage collection.
- Garbage collection freeing memory automatically for objects which are no longer in use.

28) Shallow copy & deep copy?

→ Deep - Creates new compound object before inserting copies of the items found in the original into it in a recursive manner.

shallow - Creates new object but retains references to the objects contained within original, it only copies the top-level structure without duplicating nested elements.

- When to use?

• If aim is to copy only framework ~~without~~ structure, that are flat (no nested loops / list or dict) then shallow or else deep

- Problems with deep

- Recursive objects may cause recursive loops

- Deep copy copies everything it may copy too much such as data which is intended to be shared between copies.